```
<!DOCTYPE html>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>AnonDesk · UNIFEOB</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link
href="https://fonts.googleapis.com/css2?family=DM+Serif+Display:ital@0;
1&family=DM+Sans:wght@300;400;500;600&display=swap" rel="stylesheet">
<style>
  :root {
    --blue-950: #0a1628;
    --blue-900: #0f2240;
    --blue-800: #163460;
    --blue-600: #1a56db;
    --blue-500: #2563eb;
    --blue-400: #3b82f6;
    --blue-300: #93c5fd;
    --blue-100: #dbeafe;
    --blue-50:  #eff6ff;
    --white:    #ffffff;
    --gray-200: #e2e8f0;
    --gray-400: #94a3b8;
    --gray-600: #475569;
    --success:  #10b981;
    --error:    #ef4444;
    --radius:   16px;
    --transition: 0.35s cubic-bezier(0.4,0,0.2,1);
  }

  *, *::before, *::after { box-sizing: border-box; margin: 0; padding:
0; }

  html { scroll-behavior: smooth; }

  body {
    font-family: 'DM Sans', sans-serif;
    background: var(--blue-950);
    color: var(--white);
    min-height: 100vh;
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
```

```
    position: relative;
    overflow: hidden;
  }

  /* ── Atmospheric background ── */
  .bg-glow {
    position: fixed; inset: 0; pointer-events: none; z-index: 0;
  }
  .bg-glow::before {
    content: '';
    position: absolute;
    top: -20%; left: 50%; transform: translateX(-50%);
    width: 700px; height: 700px;
    background: radial-gradient(circle, rgba(37,99,235,0.22) 0%,
transparent 70%);
    animation: pulse-glow 8s ease-in-out infinite alternate;
  }
  .bg-glow::after {
    content: '';
    position: absolute;
    bottom: -30%; right: -20%;
    width: 500px; height: 500px;
    background: radial-gradient(circle, rgba(59,130,246,0.12) 0%,
transparent 70%);
    animation: pulse-glow 11s ease-in-out infinite alternate-reverse;
  }
  @keyframes pulse-glow {
    from { transform: translateX(-50%) scale(1); opacity: 0.7; }
    to   { transform: translateX(-50%) scale(1.15); opacity: 1; }
  }

  /* Grid overlay */
  .grid-overlay {
    position: fixed; inset: 0; pointer-events: none; z-index: 0;
    background-image:
      linear-gradient(rgba(59,130,246,0.04) 1px, transparent 1px),
      linear-gradient(90deg, rgba(59,130,246,0.04) 1px, transparent
1px);
    background-size: 48px 48px;
  }

  /* ── Layout ── */
  .page {
```

```
    position: relative; z-index: 1;
    width: 100%; max-width: 520px;
    padding: 24px 20px 40px;
    display: flex; flex-direction: column; align-items: center; gap:
32px;
  }

  /* ── Header ── */
  header {
    text-align: center;
    display: flex; flex-direction: column; align-items: center; gap:
12px;
    animation: slide-down 0.7s cubic-bezier(0.4,0,0.2,1) both;
  }
  @keyframes slide-down {
    from { opacity: 0; transform: translateY(-24px); }
    to   { opacity: 1; transform: translateY(0); }
  }

  .logo-wrap {
    width: 56px; height: 56px;
    background: linear-gradient(135deg, var(--blue-600),
var(--blue-400));
    border-radius: 18px;
    display: flex; align-items: center; justify-content: center;
    box-shadow: 0 0 0 6px rgba(59,130,246,0.15), 0 12px 32px
rgba(37,99,235,0.35);
    position: relative;
  }
  .logo-wrap svg { width: 28px; height: 28px; }

  .badge {
    display: inline-flex; align-items: center; gap: 6px;
    font-size: 11px; font-weight: 600; letter-spacing: 0.08em;
    text-transform: uppercase; color: var(--blue-300);
    background: rgba(59,130,246,0.12);
    border: 1px solid rgba(59,130,246,0.25);
    border-radius: 100px; padding: 4px 12px;
  }
  .badge::before {
    content: '';
    width: 6px; height: 6px;
    background: var(--blue-400);
```

```
    border-radius: 50%;
    animation: blink 2s ease-in-out infinite;
  }
  @keyframes blink { 0%,100%{opacity:1} 50%{opacity:0.3} }

  h1 {
    font-family: 'DM Serif Display', serif;
    font-size: clamp(2rem, 8vw, 2.6rem);
    line-height: 1.1;
    letter-spacing: -0.02em;
    color: var(--white);
  }
  h1 em {
    font-style: italic;
    background: linear-gradient(135deg, var(--blue-300),
var(--blue-400));
    -webkit-background-clip: text; -webkit-text-fill-color:
transparent;
    background-clip: text;
  }

  .subtitle {
    font-size: 14px; color: var(--gray-400); line-height: 1.6;
    max-width: 360px;
  }

  /* ── Card ── */
  .card {
    width: 100%;
    background: rgba(255,255,255,0.04);
    border: 1px solid rgba(255,255,255,0.08);
    border-radius: 24px;
    padding: 28px 24px;
    backdrop-filter: blur(20px);
    -webkit-backdrop-filter: blur(20px);
    box-shadow: 0 24px 64px rgba(0,0,0,0.35);
    animation: slide-up 0.7s cubic-bezier(0.4,0,0.2,1) 0.15s both;
  }
  @keyframes slide-up {
    from { opacity: 0; transform: translateY(28px); }
    to   { opacity: 1; transform: translateY(0); }
  }
```

```
  /* ── Anonymity guarantee ── */
  .anon-bar {
    display: flex; align-items: center; gap: 10px;
    background: rgba(16,185,129,0.08);
    border: 1px solid rgba(16,185,129,0.2);
    border-radius: 12px;
    padding: 10px 14px;
    margin-bottom: 20px;
  }
  .anon-bar svg { flex-shrink: 0; color: var(--success); }
  .anon-bar span { font-size: 12.5px; color: #6ee7b7; line-height: 1.4;
}

  /* ── Form ── */
  .field-label {
    font-size: 12px; font-weight: 600;
    text-transform: uppercase; letter-spacing: 0.07em;
    color: var(--blue-300);
    margin-bottom: 8px;
    display: block;
  }

  .textarea-wrap { position: relative; }

  textarea {
    width: 100%;
    min-height: 140px;
    background: rgba(15,34,64,0.7);
    border: 1.5px solid rgba(59,130,246,0.2);
    border-radius: 14px;
    padding: 14px 16px 40px;
    color: var(--white);
    font-family: 'DM Sans', sans-serif;
    font-size: 15px;
    line-height: 1.65;
    resize: vertical;
    outline: none;
    transition: border-color var(--transition), box-shadow
var(--transition), background var(--transition);
    caret-color: var(--blue-400);
  }
  textarea::placeholder { color: rgba(148,163,184,0.5); }
  textarea:focus {
```

```
    border-color: var(--blue-500);
    background: rgba(15,34,64,0.9);
    box-shadow: 0 0 0 4px rgba(37,99,235,0.15);
  }
  textarea.error-state { border-color: var(--error); box-shadow: 0 0 0
4px rgba(239,68,68,0.12); }

  .char-count {
    position: absolute; bottom: 10px; right: 14px;
    font-size: 11px; color: var(--gray-400);
    transition: color 0.2s;
    pointer-events: none;
  }
  .char-count.warn { color: #fbbf24; }
  .char-count.over  { color: var(--error); }

  .field-error {
    font-size: 12px; color: #fca5a5;
    margin-top: 6px; padding-left: 4px;
    display: none;
  }
  .field-error.visible { display: block; }

  /* ── Submit button ── */
  .btn-submit {
    width: 100%;
    margin-top: 16px;
    padding: 15px 24px;
    background: linear-gradient(135deg, var(--blue-600) 0%,
var(--blue-500) 100%);
    border: none; border-radius: 14px;
    color: var(--white);
    font-family: 'DM Sans', sans-serif;
    font-size: 15px; font-weight: 600;
    cursor: pointer;
    display: flex; align-items: center; justify-content: center; gap:
8px;
    position: relative; overflow: hidden;
    transition: transform 0.2s, box-shadow 0.2s, opacity 0.2s;
    box-shadow: 0 4px 20px rgba(37,99,235,0.4);
  }
  .btn-submit:hover:not(:disabled) {
    transform: translateY(-2px);
```

```
    box-shadow: 0 8px 28px rgba(37,99,235,0.5);
  }
  .btn-submit:active:not(:disabled) { transform: translateY(0); }
  .btn-submit:disabled { opacity: 0.6; cursor: not-allowed; }

  .btn-submit .btn-shimmer {
    position: absolute; inset: 0;
    background: linear-gradient(105deg, transparent 40%,
rgba(255,255,255,0.15) 50%, transparent 60%);
    transform: translateX(-100%);
    transition: transform 0.6s;
  }
  .btn-submit:hover:not(:disabled) .btn-shimmer { transform:
translateX(100%); }

  /* ── Spinner ── */
  .spinner {
    width: 18px; height: 18px;
    border: 2px solid rgba(255,255,255,0.3);
    border-top-color: white;
    border-radius: 50%;
    animation: spin 0.7s linear infinite;
    display: none;
  }
  @keyframes spin { to { transform: rotate(360deg); } }
  .loading .spinner { display: block; }
  .loading .btn-text { display: none; }

  /* ── Success / Error toast ── */
  .toast {
    position: fixed;
    top: 24px; left: 50%; transform: translateX(-50%)
translateY(-100px);
    z-index: 100;
    min-width: 280px; max-width: 90vw;
    padding: 14px 20px;
    border-radius: 14px;
    display: flex; align-items: flex-start; gap: 12px;
    font-size: 14px; font-weight: 500;
    box-shadow: 0 16px 48px rgba(0,0,0,0.4);
    transition: transform 0.5s cubic-bezier(0.34,1.56,0.64,1), opacity
0.5s;
    opacity: 0;
```

```
    pointer-events: none;
  }
  .toast.show {
    transform: translateX(-50%) translateY(0);
    opacity: 1;
    pointer-events: auto;
  }
  .toast-success {
    background: #064e3b;
    border: 1px solid rgba(16,185,129,0.35);
    color: #a7f3d0;
  }
  .toast-error {
    background: #450a0a;
    border: 1px solid rgba(239,68,68,0.35);
    color: #fca5a5;
  }
  .toast-icon { flex-shrink: 0; margin-top: 1px; }
  .toast-msg { line-height: 1.5; }

  /* ── Success overlay (full card replacement) ── */
  .success-overlay {
    display: none;
    flex-direction: column; align-items: center; justify-content:
center;
    text-align: center; gap: 16px;
    padding: 12px 0;
    animation: fade-in 0.5s ease both;
  }
  @keyframes fade-in { from{opacity:0;transform:scale(0.96)}
to{opacity:1;transform:scale(1)} }
  .success-overlay.active { display: flex; }
  .card-form.hidden { display: none; }

  .checkmark-circle {
    width: 72px; height: 72px;
    background: rgba(16,185,129,0.1);
    border: 2px solid rgba(16,185,129,0.3);
    border-radius: 50%;
    display: flex; align-items: center; justify-content: center;
    animation: pop 0.5s cubic-bezier(0.34,1.56,0.64,1) both;
  }
  @keyframes pop { from{transform:scale(0)} to{transform:scale(1)} }
```

```
  .checkmark-circle svg { color: var(--success); }

  .success-title {
    font-family: 'DM Serif Display', serif;
    font-size: 1.5rem;
    color: var(--white);
  }
  .success-sub { font-size: 13.5px; color: var(--gray-400);
line-height: 1.5; max-width: 280px; }

  .btn-new {
    margin-top: 8px;
    padding: 12px 28px;
    background: transparent;
    border: 1.5px solid rgba(59,130,246,0.4);
    border-radius: 12px;
    color: var(--blue-300);
    font-family: 'DM Sans', sans-serif;
    font-size: 14px; font-weight: 500;
    cursor: pointer;
    transition: all var(--transition);
  }
  .btn-new:hover {
    background: rgba(59,130,246,0.1);
    border-color: var(--blue-400);
    color: var(--white);
  }

  /* ── Queue badge ── */
  .queue-info {
    display: flex; align-items: center; justify-content: center; gap:
8px;
    font-size: 12px; color: var(--gray-400);
    animation: slide-up 0.7s cubic-bezier(0.4,0,0.2,1) 0.3s both;
  }
  .queue-dot { width:8px; height:8px; border-radius:50%;
background:var(--blue-400); animation: blink 2s infinite; }

  /* ── Footer ── */
  footer {
    font-size: 11px; color: rgba(148,163,184,0.4);
    text-align: center; letter-spacing: 0.04em;
    animation: slide-up 0.7s cubic-bezier(0.4,0,0.2,1) 0.4s both;
```

```
  }

  /* ── Responsive ── */
  @media (max-width: 400px) {
    .card { padding: 22px 18px; }
    h1 { font-size: 1.75rem; }
  }
</style>
</head>
<body>

<div class="bg-glow"></div>
<div class="grid-overlay"></div>

<!-- Toast notification -->
<div class="toast toast-success" id="toast" role="alert"
aria-live="polite">
  <span class="toast-icon" id="toast-icon"></span>
  <span class="toast-msg" id="toast-msg"></span>
</div>

<main class="page">

  <!-- Header -->
  <header>
    <div class="logo-wrap" aria-hidden="true">
      <svg viewBox="0 0 24 24" fill="none" stroke="white"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round">
        <path d="M21 15a2 2 0 0 1-2 2H7l-4 4V5a2 2 0 0 1 2-2h14a2 2 0 0
1 2 2z"/>
        <line x1="12" y1="8" x2="12" y2="12"/><line x1="12" y1="16"
x2="12.01" y2="16"/>
      </svg>
    </div>
    <div class="badge">Perguntas ao Vivo</div>
    <h1>Anon<em>Desk</em></h1>
    <p class="subtitle">Envie sua dúvida anonimamente. Ela aparecerá em
tempo real no display do professor.</p>
  </header>

  <!-- Card -->
  <div class="card" role="main">
```

```
    <!-- Form state -->
    <div class="card-form" id="card-form">

      <!-- Anonymity bar -->
      <div class="anon-bar" aria-label="Garantia de anonimato">
        <svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2.5" stroke-linecap="round"
stroke-linejoin="round">
          <rect x="3" y="11" width="18" height="11" rx="2" ry="2"/>
          <path d="M7 11V7a5 5 0 0 1 10 0v4"/>
        </svg>
        <span>Sua identidade não é coletada nem armazenada. Nenhum dado
pessoal é enviado.</span>
      </div>

      <!-- Textarea -->
      <label class="field-label" for="question">Sua pergunta</label>
      <div class="textarea-wrap">
        <textarea
          id="question"
          name="question"
          placeholder="Digite sua dúvida aqui... Ex: Poderia explicar
novamente o conceito de herança em orientação a objetos?"
          maxlength="500"
          aria-required="true"
          aria-describedby="field-error char-count"
        ></textarea>
        <span class="char-count" id="char-count" aria-live="polite">0 /
500</span>
      </div>
      <span class="field-error" id="field-error" role="alert">Por
favor, escreva sua pergunta antes de enviar.</span>

      <!-- Submit -->
      <button class="btn-submit" id="btn-submit" type="button"
aria-label="Enviar pergunta">
        <div class="btn-shimmer"></div>
        <span class="btn-text" id="btn-text">
          <svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2.5" stroke-linecap="round"
stroke-linejoin="round" style="margin-right:4px;vertical-align:-2px">
            <line x1="22" y1="2" x2="11" y2="13"/><polygon points="22 2
15 22 11 13 2 9 22 2"/>
```

```
          </svg>
          Enviar Pergunta
        </span>
        <div class="spinner" id="spinner" aria-hidden="true"></div>
      </button>

    </div>

    <!-- Success state -->
    <div class="success-overlay" id="success-overlay"
aria-live="assertive">
      <div class="checkmark-circle">
        <svg width="36" height="36" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2.5" stroke-linecap="round"
stroke-linejoin="round">
          <polyline points="20 6 9 17 4 12"/>
        </svg>
      </div>
      <p class="success-title">Pergunta enviada!</p>
      <p class="success-sub">Sua dúvida foi recebida e será exibida no
display do professor em instantes.</p>
      <button class="btn-new" id="btn-new" type="button">Enviar outra
pergunta</button>
    </div>

  </div>

  <!-- Queue info -->
  <div class="queue-info" id="queue-info" aria-live="polite">
    <div class="queue-dot"></div>
    <span id="queue-text">Conectando ao servidor...</span>
  </div>

  <footer>UNIFEOB · AnonDesk &copy; 2025 · Open Source</footer>

</main>

<script>
  // ── Config
──────────────────────────────────────────────────────────
  const API_URL    = '/api/perguntas';       // POST endpoint
  const WS_URL     = window.location.origin; // Socket.io server origin
  const MAX_CHARS  = 500;
```

```
  const WARN_CHARS = 400;

  // ── Elements ──────────────────────────────────────────────
  const questionEl   = document.getElementById('question');
  const charCountEl  = document.getElementById('char-count');
  const fieldErrorEl = document.getElementById('field-error');
  const btnSubmit    = document.getElementById('btn-submit');
  const btnText      = document.getElementById('btn-text');
  const spinner      = document.getElementById('spinner');
  const cardForm     = document.getElementById('card-form');
  const successOver  = document.getElementById('success-overlay');
  const btnNew       = document.getElementById('btn-new');
  const queueText    = document.getElementById('queue-text');
  const toastEl      = document.getElementById('toast');
  const toastMsgEl   = document.getElementById('toast-msg');
  const toastIconEl  = document.getElementById('toast-icon');

  let toastTimer = null;
  let queueCount = 0;

  // ── Character counter ─────────────────────────────────────
  questionEl.addEventListener('input', () => {
    const len = questionEl.value.length;
    charCountEl.textContent = `${len} / ${MAX_CHARS}`;
    charCountEl.className = 'char-count' + (len >= MAX_CHARS ? ' over'
: len >= WARN_CHARS ? ' warn' : '');
    if (len > 0) {
      questionEl.classList.remove('error-state');
      fieldErrorEl.classList.remove('visible');
    }
  });

  // ── Socket.io integration (graceful fallback if not loaded)
─────────
  function initSocket() {
    if (typeof io === 'undefined') {
      queueText.textContent = 'Sistema pronto para receber perguntas.';
      return;
    }
    const socket = io(WS_URL, { transports: ['websocket'],
reconnectionDelay: 2000 });
```

```
    socket.on('connect', () => {
      queueText.textContent = 'Conectado · Sistema ao vivo';
    });
    socket.on('disconnect', () => {
      queueText.textContent = 'Reconectando...';
    });
    socket.on('queue_update', (data) => {
      queueCount = data.count ?? 0;
      queueText.textContent = queueCount === 0
        ? 'Nenhuma pergunta na fila agora.'
        : `${queueCount} pergunta${queueCount > 1 ? 's' : ''} na fila`;
    });
    socket.on('connect_error', () => {
      queueText.textContent = 'Sistema disponível · Sem tempo real';
    });
  }
  initSocket();

  // ── Submit ──────────────────────────────────────────────────────────
  btnSubmit.addEventListener('click', handleSubmit);
  questionEl.addEventListener('keydown', (e) => {
    if (e.key === 'Enter' && (e.ctrlKey || e.metaKey)) handleSubmit();
  });

  async function handleSubmit() {
    const text = questionEl.value.trim();

    // Validate
    if (!text) {
      questionEl.classList.add('error-state');
      fieldErrorEl.classList.add('visible');
      questionEl.focus();
      return;
    }
    if (text.length > MAX_CHARS) {
      showToast('error', 'A pergunta excede o limite de 500
caracteres.');
      return;
    }

    setLoading(true);
```

```
    try {
      const response = await fetch(API_URL, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ text }),
      });

      if (!response.ok) throw new Error(`Servidor retornou
${response.status}`);

      // Success
      cardForm.classList.add('hidden');
      successOver.classList.add('active');

    } catch (err) {
      console.warn('Erro ao enviar:', err);
      // Demo mode: show success anyway (for standalone preview)
      if (err.message.includes('Failed to fetch') ||
err.message.includes('NetworkError')) {
        cardForm.classList.add('hidden');
        successOver.classList.add('active');
      } else {
        showToast('error', 'Não foi possível enviar. Verifique sua
conexão e tente novamente.');
      }
    } finally {
      setLoading(false);
    }
  }

  // ── Reset form
──────────────────────────────────────────────────────

  btnNew.addEventListener('click', () => {
    questionEl.value = '';
    charCountEl.textContent = '0 / 500';
    charCountEl.className = 'char-count';
    fieldErrorEl.classList.remove('visible');
    questionEl.classList.remove('error-state');
    successOver.classList.remove('active');
    cardForm.classList.remove('hidden');
    questionEl.focus();
  });
```

```
  // ── Helpers ──────────────────────────────────────────────────────────

  function setLoading(state) {
    btnSubmit.disabled = state;
    btnSubmit.classList.toggle('loading', state);
    spinner.style.display = state ? 'block' : 'none';
    btnText.style.display = state ? 'none' : 'flex';
  }

  function showToast(type, msg) {
    clearTimeout(toastTimer);
    toastEl.className = 'toast toast-' + type;
    toastMsgEl.textContent = msg;
    toastIconEl.innerHTML = type === 'success'
      ? `<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="#34d399" stroke-width="2.5" stroke-linecap="round"
stroke-linejoin="round"><polyline points="20 6 9 17 4 12"/></svg>`
      : `<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="#f87171" stroke-width="2.5" stroke-linecap="round"
stroke-linejoin="round"><circle cx="12" cy="12" r="10"/><line x1="15"
y1="9" x2="9" y2="15"/><line x1="9" y1="9" x2="15" y2="15"/></svg>`;
    toastEl.classList.add('show');
    toastTimer = setTimeout(() => toastEl.classList.remove('show'),
4500);
  }
</script>
</body>
</html>
```